

FARTHER BRO

IT & SOFTWARE DEVELOPER PORTFOLIO

BUKHEBI BRAMWEL

“Small beginnings lead to great achievements.”

Information Technology student • Kibabii University • Kenya

02 | ABOUT FARTHER BRO

FARTHER BRO is the personal learning and professional identity of Bukhebi Bramwel. This portfolio presents an honest record of an Information Technology student's interests, foundations, practical exercises and ambitions.

Personal statement

I am interested in how technology works and how software can solve everyday problems. I am developing my knowledge step by step through university learning, independent practice and personal projects.

Professional direction

My long-term direction is software development, supported by a broad understanding of information technology, computer systems, networking, cybersecurity and digital productivity.

Personal values

- Curiosity: ask questions and keep learning.
- Discipline: practise consistently and finish tasks.
- Integrity: describe skills and experience honestly.
- Problem-solving: break difficult tasks into smaller steps.
- Growth: learn from mistakes and improve each version.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

03 | EDUCATION & LEARNING PATH

My academic foundation is Information Technology at Kibabii University. The programme introduces concepts from computing, communication, mathematics, electronics and practical use of digital tools.

Education profile

Institution: Kibabii University
Programme: Information Technology (IT)
Current stage: First Year (as recorded for this portfolio)

Core first-year study areas

- BIT1110 – Discrete Structures in IT
- BIT1120 – Information Technology Fundamentals
- BIT1130 – Fundamentals of Programming
- BIT1140 – Basic Numerical Computing
- BIT1150 – Basic Electronics
- BIT1160 – Office Automation Systems
- KUC1110 – Communication Skills
- STA1110 – Statistics and Probability

How I learn

I combine lecture notes, worked examples, practical computer exercises, revision questions and small projects. I aim to understand the reason behind each step rather than memorise commands alone.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

04 | WHAT IS INFORMATION TECHNOLOGY?

Information Technology is the use of computers, software, networks and digital systems to create, process, store, secure and communicate information.

Main components

- Hardware: physical devices such as computers, phones, servers and printers.
- Software: operating systems and applications that tell hardware what to do.
- Data: facts and records that systems process.
- Networks: connections that allow devices to exchange information.
- People and processes: users, administrators and procedures that make systems useful.

Examples in daily life

Online learning platforms, mobile banking, hospital records, business inventory systems, email, websites, digital maps and communication applications all depend on IT.

Why IT matters

IT can improve access to information, speed up work, support decision-making and connect people. It also creates responsibilities around privacy, security, accuracy and responsible use.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

05 | COMPUTER HARDWARE

Computer hardware means the physical parts of a computer system. Hardware components work together to receive input, process data, store information and produce output.

Key components

- CPU: executes instructions and performs calculations.
- RAM: temporary working memory used while programmes run.
- Storage: SSDs and hard drives keep files when power is off.
- Motherboard: connects major components.
- Power supply or battery: provides electrical power.
- Input devices: keyboard, mouse, microphone and scanner.
- Output devices: monitor, printer and speakers.

Input–process–output cycle

A user types information using a keyboard (input). The CPU and software process it. The monitor displays the result (output), while storage may save the work.

Care and maintenance

Keep devices clean and ventilated, protect them from liquid and impact, use suitable chargers, install trusted updates and back up important files.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

06 | SOFTWARE & TYPES OF SOFTWARE

Software is a set of instructions and related data that enables a computer to perform tasks. Software is commonly grouped into system software and application software.

System software

Operating systems, device drivers and system utilities manage hardware and provide a platform for applications.

Application software

Word processors, spreadsheets, browsers, media players, design tools and accounting applications help users complete particular tasks.

Programming software

Code editors, compilers, interpreters, debuggers and integrated development environments help developers create and test programmes.

Choosing software

Consider the task, compatibility, security, cost, accessibility, support and licence conditions. Download software from trusted official sources.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

07 | OPERATING SYSTEMS

An operating system (OS) manages a computer's resources and provides services for applications and users. It acts as an important link between hardware and software.

Major responsibilities

- Process management: controls running programmes.
- Memory management: allocates and releases RAM.
- File management: organises files and folders.
- Device management: coordinates peripherals through drivers.
- User interface: provides graphical or command-line interaction.
- Security: supports accounts, permissions and system protection.

Examples

Microsoft Windows, Linux distributions, macOS, Android and iOS are operating systems used on different kinds of devices.

Good practice

Use supported operating-system versions, install security updates, use a screen lock and avoid running unknown files with administrator privileges.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

08 | FILES, FOLDERS & COMMAND LINE

A file stores information, while a folder helps organise files. Understanding file paths, extensions and basic command-line operations is useful for everyday IT work.

File basics

A filename often has an extension such as .txt, .pdf, .jpg, .docx, .xlsx, .html or .c. The extension indicates the file format, but it does not guarantee that a file is safe.

Organisation

- Use clear folder names and meaningful filenames.
- Keep related project files together.
- Back up important work to a separate safe location.
- Avoid deleting system files when unsure what they do.

Command-line examples

Windows Command Prompt examples:
`dir` – list files and folders
`cd foldername` – move into a folder
`mkdir projects` – create a folder
`ipconfig` – display network configuration

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

09 | PROGRAMMING FUNDAMENTALS

Programming is the process of designing instructions that a computer can execute. A programme should solve a clearly understood problem and produce the expected result.

Basic development cycle

- Understand the problem and requirements.
- Plan the solution using steps, pseudocode or a flowchart.
- Write the code in a programming language.
- Compile or interpret and run the programme.
- Test the output and correct errors.
- Document and improve the solution.

Core concepts

Variables store values; data types describe the kind of value; operators perform actions; conditions make decisions; loops repeat instructions; functions organise reusable operations.

Good programming habits

Use meaningful names, keep code readable, test small parts, explain complex logic with comments and avoid copying code you do not understand.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

10 | C PROGRAMMING FOUNDATIONS

C is a general-purpose programming language often used to teach programming concepts and to build software where performance and low-level control are important.

Typical structure

A simple C programme includes header files, a main function, statements and a return value.

Example

```
#include <stdio.h>
int main(void) {
  printf("Hello, FARTHER BRO!\n");
  return 0;
}
```

Concepts to practise

- Variables such as int, float and char.
- Input and output using scanf and printf.
- Arithmetic operators such as +, -, * and /.
- Conditions using if and else.
- Loops using for, while and do-while.
- Functions, arrays and pointers as learning progresses.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

11 | ALGORITHMS & FLOWCHARTS

An algorithm is a finite, ordered set of steps for solving a problem. A good algorithm is clear, effective and produces the intended result for valid inputs.

Problem-solving example

To calculate the area of a rectangle: read length and width; multiply them; display the area.

Pseudocode

```
START
INPUT length, width
area = length * width
OUTPUT area
STOP
```

Common flowchart symbols

- Oval: start or end.
- Rectangle: processing step.
- Parallelogram: input or output.
- Diamond: decision such as yes/no.
- Arrow: direction of program flow.

Why plan first?

Planning helps identify missing steps, special cases and possible errors before code is written.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

12 | WEB DEVELOPMENT: HTML

HTML (HyperText Markup Language) provides the structure and meaning of web pages. It uses elements, commonly written with opening and closing tags.

Basic page structure

```
<!doctype html>
<html lang="en">
<head>
  <title>FARTHER BRO</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>My portfolio website.</p>
</body>
</html>
```

Useful HTML elements

- h1–h6: headings.
- p: paragraphs.
- a: links to pages or resources.
- img: images with alternative text.
- ul, ol and li: lists.
- section, nav, header, main and footer: semantic page structure.

Accessibility

Use meaningful headings, descriptive link text, alternative text for informative images and a logical reading order.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

13 | WEB DEVELOPMENT: CSS

CSS (Cascading Style Sheets) controls the visual presentation of web pages, including colour, typography, spacing, layout and responsiveness.

CSS example

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  line-height: 1.6;
}
h1 { color: #10264A; }
```

Important concepts

- Selectors identify which HTML elements to style.
- Properties describe the style to change.
- The box model includes content, padding, border and margin.
- Flexbox and Grid help arrange page layouts.
- Media queries adapt layouts to different screen sizes.

Design principles

Use readable contrast, consistent spacing, a small set of complementary colours and responsive layouts that work on phones, tablets and computers.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

14 | JAVASCRIPT & INTERACTIVE WEBSITES

JavaScript adds behaviour to web pages. It can respond to user actions, validate forms, update content and communicate with web services.

Common uses

- Responding to button clicks.
- Checking form fields before submission.
- Showing or hiding page sections.
- Updating content without reloading the page.
- Fetching data from an API when appropriate.

Example

```
const button = document.querySelector("button");
button.addEventListener("click", () => {
  alert("Welcome to FARTHER BRO!");
});
```

Safe development

Validate inputs, avoid inserting untrusted content as HTML, protect private information and test interactive features on different devices.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

15 | DATABASE FUNDAMENTALS

A database is an organised collection of data. Database management systems (DBMS) help users store, retrieve, update and protect information.

Relational database ideas

- A table stores related records.
- A row represents one record.
- A column represents an attribute.
- A primary key uniquely identifies a row.
- A foreign key links related tables.

Example

A student database might contain StudentID, StudentName, Programme and YearOfStudy. A separate course table could store CourseCode and CourseTitle.

Why design matters

Good database design reduces unnecessary duplication, improves consistency and makes information easier to query and maintain.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

16 | SQL BASICS

SQL (Structured Query Language) is widely used to work with relational databases. It can define tables and retrieve or modify records.

Common SQL commands

```
SELECT * FROM Students;
INSERT INTO Students (StudentID, StudentName)
VALUES (1, 'Example Student');
UPDATE Students SET YearOfStudy = 2
WHERE StudentID = 1;
DELETE FROM Students WHERE StudentID = 1;
```

Important caution

UPDATE and DELETE statements should use a correct WHERE condition. Without it, more records than intended may be changed or deleted.

Further topics

Filtering with WHERE, sorting with ORDER BY, grouping with GROUP BY, joining tables and using constraints are useful next steps.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

17 | COMPUTER NETWORKING

A computer network connects devices so that they can exchange data and share resources such as files, printers and internet access.

Network types

- PAN: personal area network.
- LAN: local area network, such as a home or office.
- MAN: metropolitan area network.
- WAN: wide area network spanning large geographical areas.

Network devices

A switch connects devices within a local network. A router forwards traffic between networks. An access point provides wireless connectivity. A modem connects to an internet service technology.

Network performance

Speed, latency, reliability, interference, congestion and hardware quality can all affect the user experience.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

18 | IP ADDRESSING & DNS

An IP address identifies a network interface for communication using Internet Protocol. Devices need suitable addressing and routing information to communicate across networks.

IPv4 and IPv6

IPv4 addresses are commonly written as four decimal numbers separated by dots, for example 192.168.1.10. IPv6 uses a longer hexadecimal format to provide a much larger address space.

Static and dynamic addressing

A static address is configured to remain consistent. A dynamic address is commonly assigned automatically using DHCP and may change over time.

DNS

The Domain Name System translates human-readable domain names into network addresses used to reach internet services.

Useful Windows command

`ipconfig /all` displays detailed network configuration, including IP address, gateway and DNS information.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

19 | INTERNET, HTTP & WEB HOSTING

The internet is a global network of connected networks. The World Wide Web is a service on the internet that uses web technologies to publish and access linked resources.

Web request basics

A browser requests a resource from a web server. HTTP defines how requests and responses are exchanged. HTTPS protects the connection using encryption through TLS.

Domain and hosting

A domain name is a human-friendly address. Web hosting provides a place for website files or application services to run.

GitHub Pages project

My FARTHER BRO portfolio is published as a GitHub Pages project site. The public address is https://wanjalabukhebi-pixel.github.io/Farther-bro-portfolio-/

Publishing checklist

- Check filenames and links carefully.
- Use the correct index.html entry page.
- Test the site on a phone and computer.
- Keep a backup of working files.
- Do not publish passwords, private keys or confidential information.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

20 | CYBERSECURITY FUNDAMENTALS

Cybersecurity is the practice of protecting devices, systems, networks and information from unauthorised access, damage and disruption.

The CIA triad

- Confidentiality: only authorised people can access information.
- Integrity: information remains accurate and is changed only appropriately.
- Availability: systems and information are accessible when needed.

Common threats

Phishing, malware, weak passwords, social engineering, unpatched software, data theft and unsafe downloads are common security risks.

Personal protection

Use unique passwords, multi-factor authentication where available, software updates, trusted downloads, backups and careful link checking.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

21 | FIREWALLS & ENCRYPTION

A firewall helps control network traffic according to security rules. Encryption transforms readable information into a protected form that can be understood only with the correct key.

Inbound and outbound rules

Inbound rules control traffic attempting to reach a device or service. Outbound rules control traffic leaving the device. Rules should allow only the traffic needed for legitimate work.

Encryption in practice

HTTPS helps protect data in transit between a browser and a website. Device encryption can help protect stored data if a device is lost or stolen.

Security mindset

Do not disable security controls just to make an application work. Understand the reason for a rule and seek trusted guidance before changing system settings.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

22 | SOFTWARE ENGINEERING

Software engineering applies systematic methods to the design, development, testing, deployment and maintenance of software.

Software development life cycle

- Requirements: understand what users need.
- Design: plan the system and its components.
- Implementation: write and integrate code.
- Testing: check that it works and handles errors.
- Deployment: make it available to users.
- Maintenance: fix issues and improve the product.

Quality attributes

Useful software should be reliable, secure, maintainable, usable, efficient and appropriate for its intended users.

Teamwork

Clear communication, version control, code review, issue tracking and documentation help teams coordinate their work.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

23 | TESTING & DEBUGGING

Testing checks whether software behaves as expected. Debugging is the process of finding, understanding and correcting the cause of a problem.

Testing levels

- Unit testing checks small functions or components.
- Integration testing checks how components work together.
- System testing checks the complete application.
- Acceptance testing checks whether user needs are met.

Debugging approach

Reproduce the problem, read the error message, isolate the relevant code, inspect values, make one change at a time and retest.

Test cases

For a calculation programme, test normal values, zero values, boundary values and invalid input. Compare actual output with expected output.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

24 | VERSION CONTROL & GITHUB

Version control records changes to project files over time. Git is a widely used version-control system, while GitHub hosts Git repositories and provides collaboration features.

Core terms

- Repository: a project folder tracked by Git.
- Commit: a saved snapshot of changes.
- Branch: an independent line of development.
- Merge: combining changes from branches.
- README: a document describing the project.

Typical workflow

```
git status
git add .
git commit -m "Update portfolio"
git push
```

Why use version control?

It helps track progress, recover earlier versions, collaborate safely and understand what changed between updates.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

25 | OFFICE AUTOMATION & DIGITAL PRODUCTIVITY

Office automation uses digital tools to create documents, organise information, analyse data and communicate more efficiently.

Common application types

- Word processors create letters, reports and notes.
- Spreadsheets organise data, use formulas and create charts.
- Presentation software communicates ideas visually.
- Database tools store structured records.
- Email and calendars support communication and planning.

Spreadsheet basics

A spreadsheet contains rows, columns and cells. Formulas begin with an equals sign. Functions such as SUM, AVERAGE, MIN and MAX help calculate results.

Professional document habits

Use clear headings, consistent formatting, accurate spelling, meaningful filenames and suitable file formats such as PDF when sharing a final document.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

26 | MATHEMATICS, LOGIC & DISCRETE STRUCTURES

Mathematics and discrete structures provide tools for reasoning about computing problems, data, algorithms and digital systems.

Sets

A set is a well-defined collection of distinct objects. Common operations include union, intersection, difference and complement.

Logic

Propositions are statements that can be true or false. Logical operators include AND, OR and NOT. Truth tables show the result of logical combinations.

Functions and relations

Functions map each input to exactly one output. Relations describe connections between elements. Properties such as reflexive, symmetric and transitive help classify relations.

Why this matters

Logical reasoning supports programming conditions, database queries, algorithm design, digital circuits and formal verification.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

27 | BASIC ELECTRONICS & DIGITAL SYSTEMS

Electronics studies the movement and control of electric charge in components and circuits. It provides foundations for understanding computer hardware and digital devices.

Basic quantities

Voltage is electrical potential difference, current is the rate of flow of charge, and resistance opposes current. Ohm's law is $V = I \times R$.

Components

Resistors limit current, capacitors store electric charge, diodes allow current primarily in one direction, and transistors can act as switches or amplifiers.

Digital logic

Digital circuits represent information using binary states, commonly 0 and 1. Logic gates such as AND, OR and NOT combine these states to produce outputs.

Safety

Electrical work should be performed only with appropriate training, safe equipment and supervision. Never experiment with mains electricity.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

28 | PROJECT SHOWCASE & PRACTICAL EXPERIENCE

This section records practical work that demonstrates learning in progress. Projects should be updated with screenshots, source links, dates and a short explanation as they develop.

Project 1 – FARTHER BRO Portfolio Website

Purpose: introduce my identity, education, interests and work. Tools and platform: HTML/CSS learning and GitHub Pages publishing. Planned improvements: profile, skills, projects, contact section, IT blog and downloadable PDF portfolio.

Project 2 – Cylinder Volume Calculator

Purpose: practise basic C programming. Inputs: cylinder diameter and height. Process: calculate radius as diameter ÷ 2, then volume as π × radius × radius × height. Output: display the volume to two decimal places.

Evidence to add over time

- A screenshot of each project running.
- A link to the source-code repository where appropriate.
- A short description of the problem and solution.
- The tools and concepts used.
- What I learned and what I would improve next.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

29 | CAREER GOALS & PROFESSIONAL DEVELOPMENT

My goal is to build a strong foundation and progress towards creating useful, secure and dependable software. I understand that professional ability comes from continued learning and practical experience.

Short-term goals

- Strengthen programming fundamentals and problem-solving.
- Complete small web and programming projects.
- Practise Git and document project work.
- Improve communication, teamwork and technical writing.

Long-term goals

- Develop deeper software engineering skills.
- Build applications that address real user needs.
- Learn testing, secure coding and deployment practices.
- Build a credible portfolio of completed projects.

Professional conduct

Respect user privacy, protect confidential information, acknowledge other people's work, follow software licences and communicate honestly about experience and capability.

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.

30 | CONTACT, VISION & CLOSING

Thank you for viewing the FARTHER BRO IT and Software Developer Portfolio. This document is a snapshot of my learning journey and will grow as I complete more coursework and practical projects.

Online portfolio

https://wanjalabukhebi-pixel.github.io/Farther-bro-portfolio-/

Professional enquiries

Please use the current contact links published on my portfolio website.

My commitment

I am committed to learning, practising and improving. Every project is an opportunity to understand technology more deeply and to create something useful.

Personal motto

“Small beginnings lead to great achievements.”

Learning note: This section records knowledge and skills being developed through study and practice. Practical ability grows through repeated exercises and completed projects.